

get_endpoint_config

REFERENCE

API consumer

read back the reader's MQTT endpoint configuration and saved endpoint names

Overview

`get_endpoint_config` reads back the reader's MQTT endpoint configuration — a faithful view of what was provisioned with `config_endpoint`. It has two modes, chosen by whether you send `endpointDetails.endpointName`:

- All-endpoints (omit `endpointDetails`):** returns every **active** endpoint's full configuration under `endpointResponse.activeEndpoints.epConfig[]`, **plus** the names of **all saved** endpoints under `endpointResponse.savedEndpoints.epNames[]`.
- Single-endpoint (send `endpointDetails.endpointName`):** returns just that one endpoint's configuration.

It is a read-only command on the device-management (MGMT) endpoint.

The response includes broker credentials (`mqttParams.password`, and `username` where set) — treat it as sensitive. See the warning in [Notes and Caveats](#). The examples on this page are the contract's, and carry placeholders rather than real credentials.

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	READ-ONLY
Required Request Fields	<code>command</code> , <code>requestId</code> (plus optional <code>endpointDetails.endpointName</code>)
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	v1.1 (API Version Reporting)
Related	<code>config_endpoint</code> , <code>get_wifi</code> , <code>get_eth</code> , <code>config_events</code>

When to Use

Use `get_endpoint_config` to inspect or audit the reader's MQTT wiring:

- **Before** `config_endpoint` — review the current configuration so you do not overwrite an active endpoint unexpectedly.
- **Confirm the plumbing** — check `url` / `port`, `protocol`, `verificationType`, and the `publishTopics` / `subscribeTopics` a broker will see.
- **List saved endpoints** — read `savedEndpoints.epNames[]` to see everything saved on the device (a superset of the active endpoints) before activating or deleting one.
- **Audit routing & security** — review each endpoint's `eventConfiguration` and `securityParams`.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

The `publishTopics` / `subscribeTopics` inside the response are **base** topics (e.g. `MDM/clients/cmnd`); on-wire they are wrapped to `<tenantId>/<topic>/<serial>`.

Credentials are presented once at CONNECT. The `requestId` you send is echoed in the response so you can pair reply to request. The endpoint's `mqttParams` govern QoS and retain, not this command.

See [Topics and Channels](#).

Key Fields

- `endpointDetails.endpointName` (request) — omit for all endpoints; supply to target one (case-sensitive).
- `activeEndpoints.epConfig[]` (response) — the full config of each **active** endpoint. Inactive (saved-but-not-activated) endpoints do **not** appear here. An endpoint appears here as soon as it is stored as active, which does not show that it connected: wait for a `CONNECTED` `mqttConnEVT` on the endpoint's connection-state topic. See [Saving an endpoint does not connect it](#).

- `savedEndpoints.epNames[]` (response) — the names of **all** saved endpoints (active *and* inactive) — the active/saved distinction in one field.
- `configuration.mqttParams.password` (response) — ⚠ a broker credential is present in the response; see [Response Payload](#) and the credential warning below.
- `response.code` (response) — read this to determine success vs failure (see [Response Codes](#)).

Request Payload

To read **all** endpoints, send just `command` and `requestId`. Use the **Schema** tab for the full field reference and the **Example** tab for the all-endpoints request.

Scope

Active endpoint config ▾

Request payload

Example: all endpoints

```
1  {
2      "command" : "get_endpoint_config" ,
3      "requestId" : "abc123"
4  }
```

Request payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	get_endpoint_config	Fixed command name.
2	requestId *	string	—	123	A unique identifier for the request; echoed back in the response so you can pair reply to request.
3	endpointDetails	object	—	—	Optional selector. Omit it to retrieve every active endpoint plus all saved names (all-endpoints mode); include it to retrieve one endpoint (single-endpoint mode).

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
4	en dp oi nt Na me	string	—	ctrl	The saved endpoint name to retrieve. Case-sensitive. When present, only that endpoint is returned.

Request Semantics

- **Modes.** All-endpoints (omit `endpointDetails`) returns every active endpoint plus all saved names; single-endpoint (`endpointDetails.endpointName`) returns just that one endpoint.

Response Payload

On success the device returns the envelope (`command` , `requestId` , `apiVersion` , `response`) with the endpoint payload under `endpointResponse` — `activeEndpoints.epConfig[]` (full configs) and `savedEndpoints.epNames[]` (all saved names). The **Example** tab shows two active endpoints — an `MDM` endpoint and a `DATA1` endpoint.

`mqttParams.password` and `username` in the example below are the broker credentials a read-back carries. The payload is taken verbatim from the contract, so it follows the contract's own placeholder policy rather than this page's: `password` reads `xxxxxxx` , a placeholder, but `username` reads `mqttuser` — a value the contract ships unmasked. Treat both as placeholders, not as credentials to reuse.

Response payload

Example: all active endpoints

```
1      {
2        "command": "get_endpoint_config",
3        "requestId": "abc123",
4        "apiVersion": "V1.22",
5        "endpointResponse": {
6          "activeEndpoints": {
7            "epConfig": [
8              {
9                "configuration": {
10                  "endpointName": "mdm_ep2",
11                  "epType": "MDM",
12                  "protocol": "MQTT",
13                  "activate": true,
14                  "url": "17L11-DS3362",
15                  "verificationType": "VERIFY_NONE",
16                  "port": 1883,
17                  "qosCommon": 0,
18                  "tenantId": "zebra",
19                  "mqttParams": {
20                    "keepAlive": 96,
21                    "cleanSession": true,
22                    "reconnectDelayMin": 5,
23                    "reconnectDelayMax": 512,
24                    "clientId": "",
25                    "username": "mqttuser",
26                    "password": "xxxxxx",
27                    "publishTopics": [
28                      {
29                        "topic": "MDM/clients/resp",
30                        "qos": 0,
31                        "retain": false
32                      },
33                      {
34                        "topic": "MDM/clients/event",
35                        "qos": 0,
36                        "retain": false
37                      },
38                      {
39                        "topic": "MDM/clients/rfid",
40                        "qos": 0,
41                        "retain": true
42                      }
43                    ],
44                    "subscribeTopics": [
45                      {
46                        "topic": "MDM/clients/cmnd",
47                        "qos": 0
```

```

48         }
49     ]
50 },
51 "securityParams": {
52     "format": "PEM",
53     "caCertificateFile": "",
54     "clientCert": "",
55     "clientKey": ""
56 },
57 "eventConfiguration": {
58     "antenna": false,
59     "terminalConnection": false,
60     "firmwareUpdate": true,
61     "gpi": false,
62     "network": true,
63     "exceptions": false,
64     "ntp": true,
65     "userApp": false,
66     "heartbeat": false,
67     "power": true,
68     "battery": true,
69     "temperature": false,
70     "fileDownload": true,
71     "cpuUsage": false,
72     "flashUsage": false,
73     "ramUsage": false,
74     "cpuThreshold": 0,
75     "ramThreshold": 0,
76     "flashThreshold": 0,
77     "temperatureThreshold": 0
78 }
79 }
80 },
81 {
82     "configuration": {
83         "endpointName": "data1_ep",
84         "epType": "DATA1",
85         "protocol": "MQTT",
86         "activate": true,
87         "url": "17L11-DS3362",
88         "verificationType": "VERIFY_NONE",
89         "port": 1883,
90         "qosCommon": 0,
91         "tenantId": "zebra",
92         "mqttParams": {
93             "keepAlive": 96,
94             "cleanSession": true,
95             "reconnectDelayMin": 5,
96             "reconnectDelayMax": 512,
97             "clientId": "",
98             "username": "mqttuser",
99             "password": "xxxxxxx",

```

```

100         "publishTopics" : [
101             {
102                 "topic" : "DATA/clients/unused" ,
103                 "qos" : 0 ,
104                 "retain" : false
105             },
106             {
107                 "topic" : "DATA/clients/dataevent" ,
108                 "qos" : 1 ,
109                 "retain" : false
110             }
111         ],
112         "subscribeTopics" : [ ]
113     },
114     "securityParams" : {
115         "format" : "PEM" ,
116         "caCertificateFile" : "" ,
117         "clientCert" : "" ,
118         "clientKey" : ""
119     },
120     "eventConfiguration" : {
121         "antenna" : false ,
122         "terminalConnection" : false ,
123         "firmwareUpdate" : true ,
124         "gpi" : false ,
125         "network" : true ,
126         "exceptions" : false ,
127         "ntp" : true ,
128         "userApp" : false ,
129         "heartbeat" : false ,
130         "power" : true ,
131         "battery" : true ,
132         "temperature" : false ,
133         "fileDownload" : true ,
134         "cpuUsage" : false ,
135         "flashUsage" : false ,
136         "ramUsage" : false ,
137         "cpuThreshold" : 0 ,
138         "ramThreshold" : 0 ,
139         "flashThreshold" : 0 ,
140         "temperatureThreshold" : 0
141     },
142     "batching" : {
143         "reportingInterval" : 1000 ,
144         "maxPayloadSizePerReport" : 1024
145     }
146 }
147 }
148 ]
149 },
150 "savedEndpoints" : {
151     "epNames" : [

```

```

152         "mdm_ep" ,
153         "mdm_ep2" ,
154         "ctrl_ep" ,
155         "data1_ep" ,
156         "mgmt_ep"
157     ]
158 }
159 },
160 "response" : {
161     "code" : 0 ,
162     "description" : "Success"
163 }
164 }

```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
1	command *	string	—	get_endpoint_config
2	requestId *	string	—	123
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1
4	endpointResponse	object	—	—
5	activeEndpoints	object	—	—
6	epConfig	array	—	—
7	configuration	object	—	—
8	endpointName	string	—	MDM
9	epType *	enum	MGMT MGMT_EVT CTRL DATA1 DATA2 MDM	MGMT

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
10	protocol *	enum	MQTT MQTT_TLS	MQTT
11	activate *	boolean	—	true
12	url *	string	—	broker.example.com
13	verification * Type	enum	VERIFY_NONE VERIFY_HOST_AND_PEER	VERIFY_HOST_AND_PEER
14	port *	integer	—	1883
15	qosCommon	integer	—	1
16	tenantId	string	—	ZEBRA
17	mqttParams	object	—	—
18	keepAlive	integer	max 65535	300
19	cleanSession	boolean	—	true
20	reconnectDelayMin	integer	—	5
21	reconnectDelayMax	integer	—	512
22	clientId	string	—	zebra-rfd40-01

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
23	username	string	—	mqttuser
24	password	string	—	xxxxxx
25	publishTopics	array	—	—
26	topic	string	—	zebra/rfd40/01/data
27	qos*	integer	—	1
28	retain*	boolean	—	false
29	subscriptions	array	—	—
30	topic*	string	—	zebra/rfd40/01/data
31	qos*	integer	—	0

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
32	reta [*] in	boolean	—	—
33	securityParams	object	—	—
34	format [*]	enum	PEM PFX	PEM
35	algorithm	enum	RSA256 RSA512	RSA256
36	caCertificateFile	string	—	cloud-trusted-ca-certs.crt
37	clientCert	string	—	client_cert.pem
38	clientKey	string	—	client_key.pem
39	eventConfiguration	object	—	—
40	antenna	boolean	—	false
41	terminalConnection	boolean	—	false
42	firmwareUpdate	boolean	—	true

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
43	gpi	boolean	—	false
44	network	boolean	—	true
45	exceptions	boolean	—	false
46	ntp	boolean	—	true
47	userApp	boolean	—	false
48	heartbeat	boolean	—	false
49	power	boolean	—	true
50	battery	boolean	—	true
51	temperature	boolean	—	false
52	fileDownload	boolean	—	true
53	cpuUsage	boolean	—	false
54	flashUsage	boolean	—	false
55	ramUsage	boolean	—	false
56	heartbeatConfiguration	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
57	interval	integer	max 65535	100
58	inventoryStatus	boolean	—	true
59	batteryStatus	boolean	—	true
60	userApps	boolean	—	—
61	cpuThreshold	integer	—	0
62	ramThreshold	integer	—	0
63	flashThreshold	integer	—	0
64	temperatureThreshold	integer	—	0

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
65	batching	object	—	—
66	reportingInterval	integer	1000–60000	5000
67	maxPayloadSizePerReport	integer	1024–5120	1024
68	savedEndpoints	object	—	—
69	epNames	array	—	["mdm_ep", ...]
70	response *	object	—	—
71	code *	integer	0–101	0
72	description *	string	—	Success

Response Semantics

Field Support. Each endpoint's `configuration.eventConfiguration` selects which events that endpoint emits. The contract declares the full set below, **but only 7 of the boolean toggles are supported today**; the remaining 13 keys are accepted by the firmware and returned in the read-back but **have no effect yet** (reserved for a future release). Enabling a reserved toggle silently does nothing.

Field	Type	Status	Description
<code>terminalConnection</code>	boolean	✓ Supported	Terminal connect/disconnect events.
<code>firmwareUpdate</code>	boolean	✓ Supported	Firmware-update events.

Field	Type	Status	Description
<code>network</code>	boolean	 Supported	Network events.
<code>heartbeat</code>	boolean	 Supported	Periodic heartbeat events.
<code>power</code>	boolean	 Supported	Power-source events.
<code>battery</code>	boolean	 Supported	Battery events.
<code>temperature</code>	boolean	 Reserved	Declared in the schema, but no temperature alert is emitted on V1.1 — the toggle is stored and read back, and does nothing.
<code>fileDownload</code>	boolean	 Supported	File-download events.
<code>antenna</code>	boolean	 Reserved	Not yet functional.
<code>gpi</code>	boolean	 Reserved	Not yet functional.
<code>exceptions</code>	boolean	 Reserved	Not yet functional.
<code>ntp</code>	boolean	 Reserved	Not yet functional.
<code>userApp</code>	boolean	 Reserved	Not yet functional.
<code>cpuUsage</code>	boolean	 Reserved	Not yet functional.
<code>flashUsage</code>	boolean	 Reserved	Not yet functional.
<code>ramUsage</code>	boolean	 Reserved	Not yet functional.

Field	Type	Status	Description
<code>cpuThreshold</code>	integer	 Reserved	Not yet functional (paired with <code>cpuUsage</code>).
<code>ramThreshold</code>	integer	 Reserved	Not yet functional (paired with <code>ramUsage</code>).
<code>flashThreshold</code>	integer	 Reserved	Not yet functional (paired with <code>flashUsage</code>).
<code>temperatureThreshold</code>	integer	 Reserved	Not yet functional.
<code>heartbeatConfiguration</code>	object	 Conditional	<code>{ interval, inventoryStatus, batteryStatus, userApps }</code> . Returned only after <code>config_events</code> enables the heartbeat (not by default). <code>userApps</code> is not currently functional.

The **7 supported** booleans are exactly the ones you can rely on today; configure event routing with `config_events`.

- **Active vs saved.** `activeEndpoints.epConfig[]` contains a full configuration only for endpoints with `activate: true`. `savedEndpoints.epNames[]` lists **every** saved endpoint — so a name that appears in `epNames` but not in `epConfig[]` is saved but inactive.
- `verificationType` **reads back as you sent it.** The reader returns the value you set with `config_endpoint`, such as `VERIFY_HOST_AND_PEER`; a plain-MQTT endpoint reads back `VERIFY_NONE`. Because one enum governs both directions, you can echo a read-back value in a later `update`.
- **Topic items and the `protocol` literal.** Each `publishTopics` / `subscribeTopics` entry carries `topic`, `qos`, and `retain`. The contract requires `qos` and `retain` on a publish item, and `topic`, `qos` and `retain` on a subscribe item — but its own read-back examples omit `retain` on every subscribe item, and an MQTT subscription has no retain flag, so do not reject a subscribe entry that lacks it. The TLS protocol literal does not differ by direction: you send `MQTT_TLS` with `config_endpoint` and read the same `MQTT_TLS` back here, so a validator can hold one enum — `MQTT` or `MQTT_TLS` — for both directions.
- **Topic slots read back in configured order.** `publishTopics` entries are returned in the order they were set with `config_endpoint`: `[0]` = command responses (`.../clients/resp`), `[1]` = events (`.../clients/event`), `[2]` = the endpoint's connection-event and **Last Will** topic (`.../clients/rfid`), with `subscribeTopics[0]` = command ingress (`.../clients/cmdnd`). A `DATA1` endpoint configured with a placeholder at `[0]` reads the tag-data topic back at `[1]`, as the contract's `DATA1` read-back shows. A read-back with a single entry is read from index `0` upward,

so that entry sits in the **reply** slot and no tag data is flowing. Match entries by **index**, not base name — the `resp` / `event` / `rfid` names are conventions.

Response Codes

`get_endpoint_config` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the endpoint configuration was retrieved.
3	Not able to retrieve information (<code>IOT_ERROR_INFO_NOT_AVAILABLE</code>) — the device could not gather the configuration at this time. Retry after a short delay; if persistent, reboot.
101	Default error — generic catch-all.

Examples

Publish `get_endpoint_config`; the device replies with the envelope shown under [Response Payload](#). The all-endpoints request (`command` + `requestId`) returns every active endpoint plus the saved-names list; the single-endpoint request (shown under [Request Payload](#)) targets one endpoint by name.

Active vs saved. In the response example, `activeEndpoints.epConfig[]` carries the full config of the two active endpoints (`mdm_ep2` and `data1_ep`), while `savedEndpoints.epNames` lists all five saved (`mdm_ep`, `mdm_ep2`, `ctrl_ep`, `data1_ep`, `mgmt_ep`) — the three that are saved but not active appear **only** as names, with no config block. When more than one endpoint is active, `epConfig[]` holds one full configuration per active endpoint.

Notes and Caveats

⚠ THE RESPONSE RETURNS BROKER CREDENTIALS

`configuration.mqttParams.password` (and `username`, when set) are part of the read-back. Redact them in logs, tickets, and screenshots; transport and store the response only over secured channels. The examples on this page carry placeholders, not real credentials.

Read `get_endpoint_config` **before** `config_endpoint` to review the current wiring and avoid clobbering an active endpoint. For the concept-level view, see [Endpoints and Topic Slots](#).

Related:  [Endpoints and Topic Slots](#) ·  [config_endpoint](#) ·  [config_events](#) ·  [get_wifi](#) ·  [Response Codes](#)

